MANANGAN, ZYRON T.

BSIT WMT IV – 2

ASSIGNMENT 01 FOR MODULE 03

Operating System 101

1. Palm OS provides no means of concurrent processing. Discuss three major complications that concurrent processing adds to an operating system.
   - Palm OS was designed as a single-tasking system with a small dynamic heap (64–256 KB) and no built-in context saving. Adding concurrency creates three major complications: (1) Memory protection, the OS must isolate per-process address spaces and manage memory reclamation without corrupting other processes; (2) Context switching, the kernel must automatically save and restore registers, stacks, and memory mappings instead of relying on applications to save their own state; and (3) Synchronization, shared resources like memory, file handles, and I/O devices require locks and semaphores to prevent race conditions and deadlocks, which Palm OS's public API lacks.

   References:

   - Usenix. (2001). The operating system. https://static.usenix.org/publications/library/proceedings/jvm01/full_papers/schneider/schneider_html/node4.html

   - Universität Hamburg. (2002). Palm OS emulator: Speicherverwaltung. https://tams.informatik.uni-hamburg.de/lehre/2002ss/proseminar/HWSW-entwurf/vortraege/03-palm-os-emulator/referat-palm-os.pdf

2. The Sun UltraSPARC processor has multiple register sets. Describe what happens when a context switch occurs if the new context is already loaded into one of the register sets. What happens if the new context is in memory rather than in a register set and all the register sets are in use?
   - The UltraSPARC uses multiple register windows pointed to by the Current Window Pointer (CWP). If the new context is already in a register set, the switch is nearly instantaneous the CWP simply moves to that window, and no memory access is needed. If the new context is in memory and all register sets are in use, the OS must first evict a victim register set by writing its contents to memory, then load the incoming context from memory into the freed set, and finally update the CWP incurring the same overhead as a conventional context switch.

   References:

   - Wikipedia. (2007). Registerfenster [Register windows]. https://de.m.wikipedia.org/wiki/Registerfenster

   - Filo. (2025, November 22). The Sun UltraSPARC processor has multiple register sets. https://askfilo.com/user-question-answers-smart-solutions/the-sun-ultrasparc-processor-has-multiple-register-sets-3332313236373933

3. When a process creates a new process using the fork() operation, which of the following state is shared between the parent process and the child process?
   a. Stack
   b. Heap

c. Shared memory segments

- c. Shared memory segments. After fork(), the child gets private copies of the stack and heap through copy-on-write, so those are not shared. Only memory segments explicitly attached as shared (e.g., via mmap with MAP_SHARED or shmop) remain visible to both processes. File descriptors are also inherited but refer to the same open file description, not the same memory.
-

References:

- Oracle. (2010). fork(2) — man pages section 2: System calls. https://docs.oracle.com/cd/E18752_01/html/816-5167/fork-2.html

- Ubuntu. (2020). fork(3posix) — Ubuntu manpage. https://manpages.ubuntu.com/manpages/focal/man3/fork.3posix.html

4. With respect to the RPC mechanism, consider the "exactly once" semantic. Does the algorithm for implementing this semantic execute correctly even if the ACK message back to the client is lost due to a network problem? Describe the sequence of messages and discuss whether "exactly once" is still preserved.
   - Yes, "exactly once" can still be preserved if the server keeps duplicate-detection state. When the client retransmits because the ACK was lost, the server checks its connection table, finds the sequence number already recorded, and returns the cached reply without re-executing the procedure. However, if the server crashes and loses that table (kept in volatile memory), it may re-execute the procedure after restart, breaking the guarantee. To survive crashes, the server must persist its completion records to durable storage (e.g., disk log) so duplicates can still be recognized after recovery.

   References:

   - Liskov, B., & Shrira, L. (1990). Providing at-most-once message delivery without connection setup. ACM. https://doi.org/10.1145/103720.103722

   - Princeton University. (2024). RPCs and failure [Lecture slides]. https://www.cs.princeton.edu/courses/archive/spring24/cos418/docs/L4-rpc-failure.pdf

5. Assume that a distributed system is susceptible to server failure. What mechanisms would be required to guarantee the "exactly once" semantics for execution of RPCs?
   - To guarantee "exactly once" despite server failures, the system needs: (1) Persistent completion records — the server durably logs each executed RPC and its reply before responding, so retransmissions after a crash return the cached reply instead of re-executing; (2) Unique request IDs — each RPC carries a client ID plus sequence number so the server can detect duplicates; (3) Server incarnation/reboot counters — written to disk and incremented on restart, so clients can detect stale replies from a previous server life; and (4) Safe garbage collection — watermark or time-based schemes to delete old completion records without opening a window for duplicate execution. Even with all these, external side effects (e.g., dispensing cash) cannot be guaranteed exactly once because the crash could occur immediately before or after the physical action.

   References:

   - Apache Software Foundation. (n.d.). RPC retry and failover design. Google Open Source. https://apache.googlesource.com/kudu/+show/c25b7cbd8d2c2746effc4b6866ea5764bcdef157/docs/design-docs/rpc-retry-and-failover.md

- Princeton University. (2024). RPCs and failure [Lecture slides]. https://www.cs.princeton.edu/courses/archive/spring24/cos418/docs/L4-rpc-failure.pdf